AutoHPFL: Autonomous Physics-Data Hybrid Power Flow Linearization Design via LLM Agents

Response Letter

Xianyang Miao[⊕], Mengshuo Jia[⊕‡⋆], Bo Yang[⊕‡⋆], and Xinping Guan[⊕‡⋆]

[⊕]*School of Automation and Intelligent Sensing, Shanghai Jiao Tong University, Shanghai 200240, China*

[‡]*The State Key Laboratory of Submarine Geoscience, Shanghai 200240, China*

[⋆]*The Key Laboratory of System Control and Information Processing, Ministry of Education of China, Shanghai 200240, China*

The authors would like to express their gratitude to the editor and reviewers for their great efforts and insightful suggestions to improve our work. We are also thankful for the encouragement from the editor and the reviewers.

In order to clearly distinguish the contents in this response letter, the comments of the editor and reviewers are provided in **blue**, the point-to-point responses are given in **black**, and the revised portions of the manuscript included here and in the revised paper are both in **red**.

Please also note that, the dotted-line hyperlinks in this response letter (e.g., **Response to Comment 1.1**) are designed as clickable links for direct navigation to the corresponding responses. However, the submission system's conversion process may disable these interactive features. For convenience, an identical version of this letter with active navigation is available by clicking here.

Response to the Editor

General Comment from the Editor to the Authors: The manuscript was reviewed by three experts in the area. Overall, the work was well-received. However, there are several issues that, in my view, can be divided into two categories. The first one concerns definitions and clarifications about nomenclature that should be improved to enhance the clarity of the manuscript ("knowledge discovery" and "algorithm design"). The second one includes questions regarding the proposed methodology that, if properly addressed, could further support the work. For example, the manuscript would benefit from a more detailed discussion of the context and scope of application of the proposed methodology. Therefore, I request that the authors carefully review the reviewers' comments and address them as thoroughly as possible. Where they choose to maintain the current approach, they should provide clear and well-supported arguments to justify their decision.

The authors must strictly adhere to the IEEE manuscript format and template. The text in the figures should be in an 8-point font size. On the first page, please include each author's name and affiliation (including city and country) at the bottom of the first column. Revisions are limited to 3.5 pages.

Response to the General Comment: We appreciate the editor's thoughtful comments and the opportunity to improve our manuscript. In response, we have revised the manuscript carefully to address the concerns raised by the reviewers and the Editor. In the revision, we clarified the positioning of AutoHPFL, refined the problem formulation and application scope, expanded the numerical results, improved the methodology description, redesigned Fig. 1, and added discussion on LLM/RAG reliability. Regarding the two main issues highlighted by the Editor, we made the following corresponding revisions.

First, to clarify the manuscript positioning, terminology, and application scope, we have supplemented the following:

- We revised the manuscript to consistently position AutoHPFL as an autonomous algorithm-design framework rather than a knowledge-discovery claim; see **Response to Comment 1.1**.
- We clarified that the scenario of this paper is power-flow linearization, rather than direct OPF or standard state estimation; see **Response to Comment 1.2** for details.
- We refined the definitions of the input/output matrices and added voltage-magnitude results to demonstrate the extensibility of the output matrix; see **Response to Comment 1.3** and **Response to Comment 2.1**.

Second, we strengthened the methodology explanation and supporting evidence. This set of revisions addresses the concerns related to the clarity of the AutoHPFL workflow, the implementation of the tree search and LLM-agent generation process, and the reliability of the reported results, as summarized below:

- We expanded the explanations of Tree/Node, MCTS, RAG, LLM-agent generation, simulator-based evaluation, and tree-informed reflection; see **Response to Comment 1.4**.
- We redesigned Fig. 1 to separate the candidate-method search tree from operations on selected parent nodes and to provide an illustrative example; see **Response to Comment 2.2**.
- We clarified the mapping ψ and the method representation used for novelty evaluation; see **Response to Comment 2.3**.

- We added technical intuition for the numerical gains, reported three independent AutoHPFL runs and their arithmetic mean, and discussed hallucination-related reliability concerns; see **Response to Comment 3.1** and **Response to Comment 3.2**.

In addition, we also checked the IEEE manuscript format and template requirements. The revised figures use at least 8-point text, the first page includes author names and affiliations with city and country, and the manuscript has been kept within the required page limit.

Response to Reviewer 1

General Comments to the Author: This letter proposes a framework that utilizes LLM agents to discover HPFL methods. This is a novel and interesting topic which merits further investigation. However, there are some ambiguities in the presentation, description, and numerical studies of the method, which require clarification. Here are my detailed comments:

Response to the General Comment: We thank the reviewer for recognizing the novelty and potential value of this work. We agree that the original manuscript contained ambiguities regarding the paper positioning, application scenario, output variable definition, and methodology details. In the revised manuscript, we clarified that AutoHPFL is positioned as an autonomous algorithm-design framework for HPFL, rather than as a claim of discovering a new universal linear power-flow formulation. We also revised the problem formulation, experimental setup, methodology description, and figures accordingly. Specifically, please refer to:

- **Response to Comment 1.1** for the distinction between knowledge discovery and algorithm design and the corresponding revisions to the Abstract, Introduction, Index Terms, and Conclusion.
- **Response to Comment 1.2** for the clarification of the application scenario, the role of OPF and state estimation, and the use of Gaussian noise.
- **Response to Comment 1.3** for the generalized definition of the output matrix $\mathbf{Y}$ and the additional voltage-magnitude results.
- **Response to Comment 1.4** for the expanded methodology description, including Tree/Node definitions, MCTS-LLM interaction, RAG knowledge sources, and the revised Fig. 1 with an illustrative example.

Comment 1.1: The authors should distinguish between "knowledge discovery" and "algorithm design," as these two differ significantly in nature and difficulty. If it is "knowledge discovery," meaning a novel linear power flow formulation has been identified using LLMs, the formulation needs to be fully presented and analyzed. Conversely, if it is "algorithm design," such as using LLMs for feature engineering or tuning, the Abstract and Introduction should be revised accordingly.

Response to Comment 1.1: We thank the reviewer for pointing out this key issue. We agree that "knowledge discovery" and "algorithm design" differ significantly in research objective and required evidence. Some expressions in the original manuscript could have been interpreted as claiming that a new universal linear power-flow formulation had been discovered by LLMs. This is not the intended contribution. The main contribution of this Letter is to propose AutoHPFL as an autonomous algorithm-design framework for searching and refining executable HPFL candidate methods, including feature mappings, partitioning rules, and regression structures.

Accordingly, we revised the Abstract, Index Terms, Introduction, and Conclusion to consistently position AutoHPFL as an algorithm-design framework. We removed or weakened expressions that could imply knowledge discovery or a new physical formulation, and replaced them with descriptions such as "algorithm design," "feature-mapping design," and "executable candidate-method search." Specifically, the Abstract, Index Terms, Introduction, and Conclusion were revised as follows:

Abstract:

This letter presents AutoHPFL, an autonomous algorithm-design framework that coordinates large language model (LLM) agents to explore and refine Physics-data-hybrid-driven power flow linearization (HPFL) methods. Rather than relying on manually crafted feature mappings, AutoHPFL autonomously searches the design space of linearization strategies through structured exploration, where agents iteratively refine candidates via tree-informed reflection on real-environment execution feedback, coordinated through novelty-aware Monte Carlo Tree Search (MCTS) to systematically identify high-performing methods beyond manual feature engineering. Experiments on IEEE 118-bus, 300-bus and 33-bus test systems demonstrate up to 35.9% error reduction over representative baselines. Moreover, the generated candidates exhibit interpretable design patterns, such as physics-guided feature construction and topology-aware local modeling, indicating the potential of LLM-agent search for autonomous power system algorithm design.

Index Term:

Power flow linearization, Physics-data-hybrid-driven methods, autonomous algorithm design, large language model agents, feature-mapping design.

Section I. Introduction:

... Instead of relying on manually crafted feature mappings, AutoHPFL formulates HPFL design as a tree-structured search process over candidate methods. Beyond HPFL, the proposed framework has the potential to be adapted to other power-system modeling and optimization tasks. The results demonstrate that AutoHPFL can automatically generate effective HPFL candidates with consistent improvements over representative baselines, indicating its potential as an autonomous algorithm-design tool for power-system linearization tasks.

Section V. Conclusion:

This letter presents AutoHPFL, an autonomous framework that coordinates LLM agents via tree-structured exploration, tree-informed reflection, and novelty-aware search to design and refine HPFL methods. Experiments on three IEEE systems show consistent improvements for both branch active-power flow and voltage-magnitude output....

Comment 1.2: The application scenario of the paper is not clearly defined. The Introduction mentions OPF and linear power flow, while the Experiments section introduces Gaussian noise, which makes it appear more like a state estimation problem. Please clarify the specific scenario of the study.

Similarly, if the LLM has indeed discovered a novel linear power flow formulation, its effectiveness should be validated across multiple downstream tasks, such as PF, OPF, and SE. If, however, it is a matter of algorithm design, the paper should be revised accordingly.

Response to Comment 1.2*:* We thank the reviewer for identifying the ambiguity in the application scenario. This Letter focuses on physics-data-hybrid-driven power-flow linearization and presents AutoHPFL as an autonomous algorithm-design framework for constructing executable HPFL candidate methods. It does not aim to directly solve OPF or standard state estimation, nor does it claim to discover a new universal linear power-flow formulation. This clarification is based on the following points:

First, a standard state-estimation problem usually takes noisy meter measurements as inputs and estimates the system states, typically voltage magnitudes and phase angles, through a measurement model. In contrast, our experiments evaluate a power-flow linearization task where the input $\mathbf{X}$ is constructed from bus active and reactive injections, and the output $\mathbf{Y}$ is the target variable to be linearized, mainly branch active-power flow $\mathbf{PF}$, with voltage magnitude $\mathbf{V}_m$ additionally evaluated as an expanded output. Therefore, although noise is introduced into the input samples, the task is not formulated as state estimation.

Second, the mention of OPF in the Introduction is intended only to motivate why fast and more accurate linearized power-flow models are useful for downstream power-system applications, rather than to indicate that OPF is the direct task evaluated in this Letter.

Third, the Gaussian noise is included to emulate data-acquisition uncertainty in the input samples, so that the evaluated setting better reflects imperfect input data that may arise in practice. It is not used to define a measurement-to-state inverse problem. The evaluation targets remain power-flow linearization outputs obtained from AC power-flow data, and the reported metrics measure the linearization errors of the HPFL candidates under this noisy-input setting.

In the revised paper, we revised the Introduction to clarify the application scope. The Introduction now defines AutoHPFL as "an autonomous algorithm design framework for physics-data-hybrid-driven power flow linearization" and further clarifies that AutoHPFL searches executable HPFL candidate methods rather than directly solving OPF or discovering a new universal linear power-flow formulation.

Section I. Introduction:

...

To address this gap, we propose AutoHPFL, an autonomous algorithm-design framework for physics-data-hybrid-driven power flow linearization, which integrates LLM-agent reflection with Monte Carlo Tree Search (MCTS) and novelty-aware exploration to search executable HPFL candidate methods. Instead of relying on manually crafted feature mappings, AutoHPFL formulates HPFL design as a tree-structured search process over candidate methods.

We also revised Section IV-A to clarify the role of Gaussian noisethe and input setting. The revised text specifies that gaussian noise is injected to emulate data-acquisition uncertainty in the input data rather than to formulate a standard state-estimation problemthe, and that the input $\mathbf{X}$ is constructed from bus active and reactive injections as well as voltage magnitude.

Section IV-A. Experiment Setup

... To emulate data-acquisition uncertainty, Gaussian noise with a signal-to-noise ratio (SNR) of 45 dB is injected into the input data. The input $\mathbf{X}$ is constructed from the bus active and reactive injections, while the outputs matrix $\mathbf{Y}$ contains branch active-power flow $\mathbf{P}_F$ and voltage magnitude $\mathbf{V}_m$....

Comment 1.3: In the paper, Y represents the branch power flow. Should it also include voltage and phase angle?

Response to Comment 1.3: We thank the reviewer for pointing out that the definition of the output variable $\mathbf{Y}$ was not sufficiently clear. In the revised paper, we clarified the generalized definition of $\mathbf{Y}$ and expanded the numerical experiments by adding bus voltage-magnitude $\mathbf{V}_m$ results. Specific revisions are provided as follows.

First, in the generalized HPFL formulation, the output matrix $\mathbf{Y}$ denotes the target power-flow quantities to be approximated by the linearized model. Depending on the downstream target, $\mathbf{Y}$ can include branch active-power flows, branch reactive-power flows, bus voltage magnitudes, bus voltage phase angles, or their combinations. The input matrix $\mathbf{X}$ denotes the known operating-condition variables available before solving the target power-flow quantities, such as bus active-power injections, bus reactive-power injections, load/generation setpoints, or other measurable operating variables. In this Letter, $\mathbf{X}$ is constructed from bus active and reactive injections. The primary output $\mathbf{Y}$ is instantiated as branch active-power flow $\mathbf{P}_F$, and bus voltage magnitude $\mathbf{V}_m$ is additionally evaluated by expanding the output matrix. Bus voltage phase angles can also be incorporated by appending the corresponding voltage-angle columns to $\mathbf{Y}$ under the same formulation, but they are not separately reported due to the page limit. Accordingly, we revised Section II to clarify the generalized meaning of $\mathbf{Y}$; we also revised Section IV-A to specify the concrete choices of $\mathbf{X}$ and $\mathbf{Y}$ used in the experiments.

Section II. Problem Formulation

Given a power-system physical descriptor $\mathcal{G}$ (e.g. topologies, branch parameters, etc.) and paired power flow datasets $\mathbf{X} \in \mathbb{R}^{N \times d_x}$ and $\mathbf{Y} \in \mathbb{R}^{N \times d_y}$, generalized HPFL is formulated as

$$\hat{\mathbf{Y}} = \Phi(\mathbf{X}, \mathcal{G})\boldsymbol{\beta}, \tag{1}$$

where $\Phi(\cdot)$ denotes a physics-data hybrid feature mapping, and $\boldsymbol{\beta}$ is the coefficient matrix. The output matrix $\mathbf{Y}$ can represent branch flows, voltage magnitudes, voltage angles, or their combinations, depending on the downstream target.

The design objective is to find an optimal pair $\Phi^*, \boldsymbol{\beta}^*$ that minimizes the mean relative error (MRE) between $\mathbf{Y}$ and $\hat{\mathbf{Y}}$:

$$(\Phi^*, \boldsymbol{\beta}^*) = \arg\min_{\Phi, \boldsymbol{\beta}} \frac{1}{N\, d_y} \sum_{i=1}^{N} \sum_{j=1}^{d_y} \frac{|\hat{Y}_{ij} - Y_{ij}|}{|Y_{ij}| + \epsilon}. \tag{2}$$

where ϵ is a small constant to avoid division by zero. Concrete choices of $\mathbf{X}$ and $\mathbf{Y}$ are specified in Section IV-A.

Section IV-A. Experimental Setup

... The input $\mathbf{X}$ is constructed from the bus active and reactive injections, while the outputs matrix $\mathbf{Y}$ contains branch active-power flow $\mathbf{P}_F$ and voltage magnitude $\mathbf{V}_m$.

Second, following the reviewer's suggestion on whether $\mathbf{Y}$ should include voltage-related quantities, we further expanded the experiments beyond branch active-power flow and added bus voltage-magnitude linearization results in the revised manuscript. This additional evaluation uses bus voltage magnitude as an alternative target output in $\mathbf{Y}$, while keeping the same AutoHPFL search framework. The complete results and for both branch active-power flow and bus voltage magnitude are reported below:

Section IV-B. Result Analysis

Table I reports PF and V_m MRE over three independent AutoHPFL runs, with the Mean row denoting their arithmetic average. Compared with the best non-AutoHPFL baseline in each column, AutoHPFL consistently achieves lower mean errors on all three systems and for both output types. In particular, AutoHPFL reduces PF MRE by up to 35.9% on case300 and V_m MRE by up to 14.7% on case33bw. Moreover, all independent AutoHPFL runs

outperform the corresponding best baseline, indicating that the gain is not due to a single favorable LLM-agent trajectory.

TABLE I. PF and voltage-magnitude linearization results (MRE, %).

(a) Branch Active-Power Flow (PF MRE)

Method	case118	case300	case33bw
LS_LIFX	1.012	1.165	0.757
RR_KPC	0.819†	0.932	0.625
LS_COD	1.397	1.169	0.559†
RR_WEI	2.583	4.464	0.561
PLS_NIP	1.397	1.317	0.559†
PLS_CLS	0.822	0.842†	0.754
DLPF_C	1.466	1.418	0.566
AutoHPFL (3 runs)	0.573 / 0.530 / 0.558	0.587 / 0.528 / 0.502	0.463 / 0.468 / 0.468
AutoHPFL Mean	**0.554**	**0.539**	**0.466**

(b) Voltage Magnitude (V_m MRE)

Method	case118	case300	case33bw
LS_LIFX	0.793	0.570†	0.770
RR_KPC	0.724	0.876	0.595
LS_COD	0.551†	0.571	0.524†
RR_WEI	2.153	12.067	0.569
PLS_NIP	0.551†	0.660	0.524†
PLS_CLS	0.718	0.798	0.709
DLPF_C	0.612	0.715	0.565
AutoHPFL (3 runs)	0.499 / 0.463 / 0.472	0.562 / 0.510 / 0.451	0.446 / 0.447 / 0.449
AutoHPFL Mean	**0.478**	**0.507**	**0.447**

† denotes the best non-AutoHPFL baseline in each column. AutoHPFL (3 runs) lists three independent runs, and Mean is their arithmetic average. Lower is better.

Please note that the above expanded results further support the generality of the output formulation. For branch active-power flow, AutoHPFL achieves lower mean errors than the best non-AutoHPFL baseline on all three systems. For bus voltage magnitude, AutoHPFL again achieves the lowest mean error on all three systems. More importantly, every independent AutoHPFL run remains below the corresponding best non-AutoHPFL baseline for both branch active-power flow and bus voltage magnitude. This indicates that the improvement is not caused by a single favorable run and that the proposed search framework can be applied to different choices of the target output matrix $\mathbf{Y}$.

Comment 1.4: Due to page limitations, the description of the methodology is not sufficiently detailed. For instance, the explanations of "Tree" and "Node," how MCTS is combined with LLM reflection, and what specific knowledge is utilized for RAG are lacking. These aspects should be described in greater detail, or an illustrative example should be provided in the figure.

Response to Comment 1.4: We thank the reviewer for pointing out that the methodology description was not sufficiently detailed. To address these concerns, we revised Section III and Fig. 1 from four aspects, including (1) the definitions of Tree and Node, (2) the division of responsibility between tree-level MCTS search and node-level LLM-agent generation, (3) the construction and use of the RAG database, and (4) the redesigned framework figure with a real selected-node generation example. The specific revisions are as follows.

First, we clarified the definitions of Tree and Node. In AutoHPFL, each node in the search tree represents an

executable HPFL candidate method. Each node stores standardized code, textual description, performance metrics, novelty score, parent link, visit count, and accumulated reward. The tree records how candidate methods evolve from root baseline methods to later child variants.

Section III-A. Framework Overview

...

Each node in the search tree represents a complete executable HPFL candidate method, including its standardized code, textual description, performance metrics, novelty score, parent link, visit count, and accumulated reward. Each edge denotes a valid transformation from a parent method to a child variant. Thus, the tree explicitly records how candidate algorithms evolve from root baseline methods.

Second, we clarified the division of responsibility between tree-level MCTS search and node-level LLM-agent generation. At the tree level, MCTS is used as the tree-level search procedure for parent-node ranking and selection, receiving simulator-based evaluation feedback, computing rewards, and backpropagating node statistics. At the node level, once parent nodes are selected by UCB, each selected parent node is assigned to an individual LLM agent to generate an executable child candidate using the parent method, RAG-retrieved context, elite-node references, and historical feedback. During evaluation, the agent assists with code execution, interface checking, log parsing, and result collection, but the final performance score is computed only from simulator outputs.

Section III-B. Tree-Level Search with Novelty-aware MCTS

Given the current method tree, the tree-level search process ranks existing nodes, selects parent nodes for generation, and updates node statistics after child evaluation. In each iteration, parent nodes are selected by Upper Confidence Bound (UCB):

$$\mathrm{UCB}_i = \bar{r}_i + c\sqrt{\frac{\ln N}{n_i}} \tag{3}$$

where $\bar{r}_i$ is the average reward of node i, N is the total visits to the parent level, n_i is the visit count of node i, and c determines the exploration-exploitation trade-off. The first term favors high-performing methods, while the second term encourages visiting less-explored branches.

Beyond standard MCTS, we design a novelty-aware reward to prevent search from collapsing to a narrow family of similar methods. The reward combines prediction accuracy with structural novelty:

$$r = -\mathrm{MRE} + \lambda \cdot s_{novelty} \tag{4}$$

where λ balances accuracy and diversity.

Section III-C. Node-Level Generation with Tree-informed Reflection

At the node level, parent nodes selected by the tree-level search are processed in parallel, with each selected node assigned to a dedicated LLM agent to generate an executable child candidate under a standardized interface. The generation is conditioned on the parent method, RAG-retrieved context, elite-node references, and historical search feedback. The RAG module provides external context from domain papers, implemented baseline methods, and code templates, and is used only as generation support rather than verified knowledge. Thus, the agent performs

structured refinement of the selected parent method instead of free-form method generation.

The generated child code is then executed by a simulator wrapper in a real computational environment. The wrapper completes necessary execution settings, runs the candidate script, extracts performance metrics from the output, and returns structured error messages when execution fails or times out. During this process, the agent assists with interface checking, log parsing, and result collection, but the final score is computed only from simulator outputs. This simulator-in-the-loop validation prevents LLM-generated candidates from being accepted based on textual plausibility alone.

If execution fails or the child degrades relative to its parent, tree-informed reflection is triggered. The agent diagnoses the failure using execution logs and parent-method structure, and retrieves cross-branch references from low-error and high-novelty elite pools maintained by the search tree. These elite nodes provide examples of effective and diverse designs from other branches. Based on this feedback, the agent regenerates a revised candidate, turning reflection into simulator-grounded error correction and cross-branch knowledge transfer.

Third, we clarified the knowledge source used by RAG. The RAG database includes HPFL/DPFL-related papers, implemented methods, method variants, standardized MATLAB code interfaces, and feature-construction templates. It contains more than 50 implemented methods and templates. We further clarified that the seven methods reported in Table II are representative baselines selected for compact numerical comparison, while the RAG and novelty-evaluation process use the full database to support candidate generation and diversity assessment.

Section III-C. Node-Level Generation with Tree-informed Reflection

...

The generated child code is then executed by a simulator wrapper in a real computational environment. The wrapper completes necessary execution settings, runs the candidate script, extracts performance metrics from the output, and returns structured error messages when execution fails or times out. During this process, the agent assists with interface checking, log parsing, and result collection, but the final score is computed only from simulator outputs. This simulator-in-the-loop validation prevents LLM-generated candidates from being accepted based on textual plausibility alone.

Fourth, we redesigned Fig. 1 and added a real illustrative example. The revised Fig. 1(a) separates the candidate-method search tree from operations on selected parent nodes, thereby clarifying the roles of Tree/Node, tree-level MCTS search, node-level LLM generation, and simulator-based evaluation. Fig. 1(b) provides a real selected-node generation example, where K-means-based piecewise ridge regression (KM-PR) is expanded into PLS-assisted K-means piecewise ridge regression (PLS-KM-PR), and illustrates how the generated child candidate is evaluated and how the feedback is used to update the selected branch.

Section III. Methodology

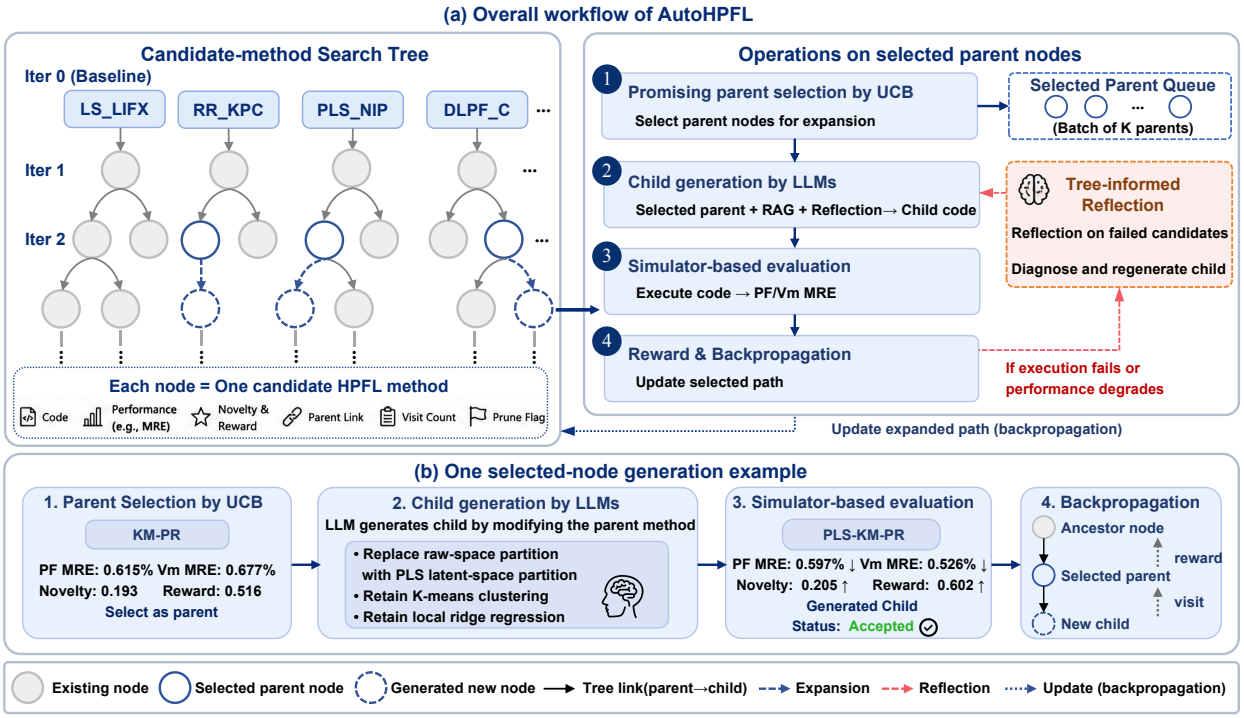

Fig. 1. Overall workflow of AutoHPFL. Panel (a) illustrates the candidate-method search tree and the operations performed on selected parent nodes. Each node in the tree represents an executable HPFL candidate method, and UCB is used to select promising parent nodes for generation. The selected parents generate child methods using LLM agents, are then evaluated through simulator-based execution, and updated through backpropagation along the selected paths. Panel (b) shows a selected-node generation example where AutoHPFL selects K-means-based piecewise ridge regression (KM-PR) as the parent method, generates PLS-assisted K-means piecewise ridge regression (PLS-KM-PR), and achieves lower PF and V_m MRE in simulator-based evaluation.

Response to Reviewer 2

Comments to the Author

This letter presents AutoHPFL, an autonomous algorithm discovery framework combining LLM agents and MCTS for power flow linearization. The overall quality of the work is good, and the proposed method demonstrates promising performance. However, several aspects should be improved for better clarity and completeness:

Response to the General Comment: We thank the reviewer for the positive assessment of the overall quality and potential of the proposed method. We carefully addressed the reviewer's comments by improving the clarity and completeness of the formulation, framework figure, space mapping, and figure labels. Specifically, please refer to:

- **Response to Comment 2.1** for the revised definition of the output matrix $\mathbf{Y}$ and the concrete experimental instantiation of $\mathbf{X}$ and $\mathbf{Y}$.
- **Response to Comment 2.2** for the redesigned Fig. 1, which separates the search tree from selected-node operations and simplifies the core data flow.
- **Response to Comment 2.3** for the implementation details of the mapping ψ, including code-text serialization, embedding, and cosine-distance novelty evaluation.
- **Response to Comment 2.4** for the correction of the label typo in Fig. 2 and the naming consistency check.

Comment 2.1: In Section II, what variable Y represents is not clear. Please clarify its meaning.

Response to Comment 2.1: We thank the reviewer for pointing out that the meaning of $\mathbf{Y}$ in Section II was unclear. To remove this ambiguity, we revised the formulation to define $\mathbf{Y}$ as the general target-output matrix of HPFL, rather than as a notation tied to one specific physical quantity. We also clarified the corresponding experimental instantiation of both $\mathbf{X}$ and $\mathbf{Y}$ in Section IV-A, and added voltage-magnitude results in the revised numerical study to show how $\mathbf{Y}$ can be expanded for different output targets. Details are as follows.

First, in the revised paper, $\mathbf{Y}$ is defined as the collection of target power-flow quantities to be approximated by the linearized model. Therefore, it may contain branch active-power flows, branch reactive-power flows, bus voltage magnitudes, bus voltage phase angles, or a combination of these quantities, depending on the intended application. The input matrix $\mathbf{X}$ is defined as the available operating-condition information, such as bus active/reactive injections, load and generation setpoints, or other measurable variables. In the experiments of this Letter, $\mathbf{X}$ is constructed from bus active and reactive injections, and $\mathbf{Y}$ is mainly instantiated as branch active-power flow $\mathbf{P}_F$. Voltage magnitude $\mathbf{V}_m$ is further included as an additional output target in the revised experiments. Voltage phase angles can be handled in the same way by appending angle-related columns to $\mathbf{Y}$, but are not separately reported because of the page limit. The revision in Section II is as follows.

Section II. Problem Formulation

Given a power-system physical descriptor $\mathcal{G}$ (e.g. topologies, branch parameters, etc.) and paired power flow datasets $\mathbf{X} \in \mathbb{R}^{N \times d_x}$ and $\mathbf{Y} \in \mathbb{R}^{N \times d_y}$, generalized HPFL is formulated as

$$\hat{\mathbf{Y}} = \Phi(\mathbf{X}, \mathcal{G})\boldsymbol{\beta}, \tag{5}$$

where $\Phi(\cdot)$ denotes a physics-data hybrid feature mapping, and $\boldsymbol{\beta}$ is the coefficient matrix. The output matrix $\mathbf{Y}$ can represent branch flows, voltage magnitudes, voltage angles, or their combinations, depending on the downstream target.

The design objective is to find an optimal pair $\Phi^*, \boldsymbol{\beta}^*$ that minimizes the mean relative error (MRE) between $\mathbf{Y}$ and $\hat{\mathbf{Y}}$:

$$(\Phi^*, \boldsymbol{\beta}^*) = \arg\min_{\Phi, \boldsymbol{\beta}} \frac{1}{N\, d_y} \sum_{i=1}^{N} \sum_{j=1}^{d_y} \frac{|\hat{Y}_{ij} - Y_{ij}|}{|Y_{ij}| + \epsilon}. \tag{6}$$

where ϵ is a small constant to avoid division by zero. Concrete choices of $\mathbf{X}$ and $\mathbf{Y}$ are specified in Section IV-A.

Section IV-A. Experimental Setup

... The input $\mathbf{X}$ is constructed from the bus active and reactive injections, while the outputs matrix $\mathbf{Y}$ contains branch active-power flow $\mathbf{P}_F$ and voltage magnitude $\mathbf{V}_m$.

Second, to make the revised definition of $\mathbf{Y}$ more concrete, we added numerical results for voltage magnitude in addition to the original branch active-power flow results. This experiment keeps the same AutoHPFL search framework but changes the target contents of $\mathbf{Y}$ to include $\mathbf{V}_m$. The revised results are shown below.

Section IV-B. Result Analysis

Table I reports PF and V_m MRE over three independent AutoHPFL runs, with the Mean row denoting their arithmetic average. Compared with the best non-AutoHPFL baseline in each column, AutoHPFL consistently achieves lower mean errors on all three systems and for both output types. In particular, AutoHPFL reduces PF MRE by up to 35.9% on case300 and V_m MRE by up to 14.7% on case33bw. Moreover, all independent AutoHPFL runs outperform the corresponding best baseline, indicating that the gain is not due to a single favorable LLM-agent trajectory.

TABLE I. PF and voltage-magnitude linearization results (MRE, %).

(a) Branch Active-Power Flow (PF MRE)			
Method	**case118**	**case300**	**case33bw**
LS_LIFX	1.012	1.165	0.757
RR_KPC	0.819†	0.932	0.625
LS_COD	1.397	1.169	0.559†
RR_WEI	2.583	4.464	0.561
PLS_NIP	1.397	1.317	0.559†
PLS_CLS	0.822	0.842†	0.754
DLPF_C	1.466	1.418	0.566
AutoHPFL (3 runs)	0.573 / 0.530 / 0.558	0.587 / 0.528 / 0.502	0.463 / 0.468 / 0.468
AutoHPFL Mean	**0.554**	**0.539**	**0.466**
(b) Voltage Magnitude (V_m MRE)			
Method	**case118**	**case300**	**case33bw**
LS_LIFX	0.793	0.570†	0.770
RR_KPC	0.724	0.876	0.595
LS_COD	0.551†	0.571	0.524†
RR_WEI	2.153	12.067	0.569
PLS_NIP	0.551†	0.660	0.524†
PLS_CLS	0.718	0.798	0.709
DLPF_C	0.612	0.715	0.565
AutoHPFL (3 runs)	0.499 / 0.463 / 0.472	0.562 / 0.510 / 0.451	0.446 / 0.447 / 0.449
AutoHPFL Mean	**0.478**	**0.507**	**0.447**

† denotes the best non-AutoHPFL baseline in each column. AutoHPFL (3 runs) lists three independent runs, and Mean is their arithmetic average. Lower is better.

Comment 2.2: In Fig. 1, the overall framework diagram is a bit cluttered. The baseline selection, horizontal iterations, and internal MCTS loops are intertwined. The core data flow is difficult to follow and should be simplified.

Response to Comment 2.2: We thank the reviewer for pointing out the readability issue in the original Fig. 1. We agree that the original figure mixed baseline selection, horizontal iterations, and internal MCTS loops, making the core data flow difficult to follow. We therefore redesigned Fig. 1 to separate the search-tree state from the algorithmic operations performed on selected parent nodes.

Specifically, the revised Fig. 1(a) separates the candidate-method search tree from operations on selected parent nodes. This helps readers distinguish the tree structure, candidate nodes, MCTS controller, LLM agents, and simulator-based evaluation. We also simplified the main workflow into parent selection, child generation, simulator-based evaluation, and reward/backpropagation. In addition, Fig. 1(b) provides a real selected-node generation example to show how a parent method is transformed into a child candidate.

Section III. Methodology

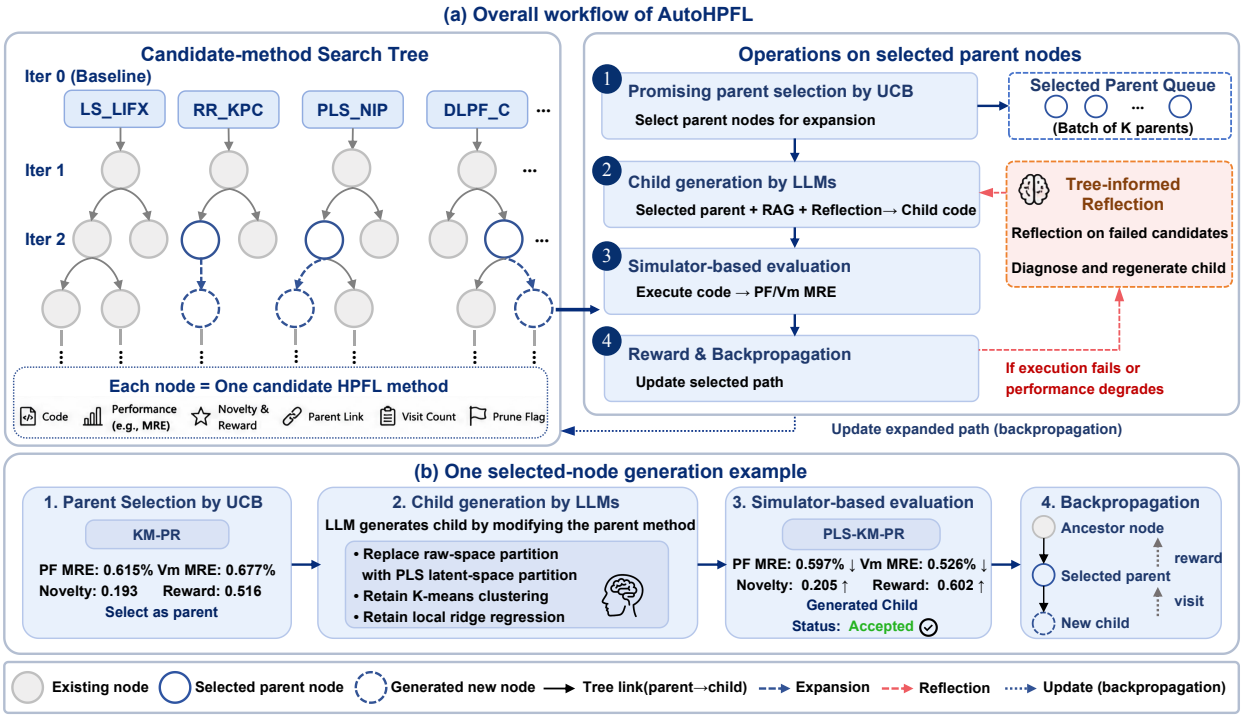

Fig. 1. Overall workflow of AutoHPFL. Panel (a) illustrates the candidate-method search tree and the operations performed on selected parent nodes. Each node in the tree represents an executable HPFL candidate method, and UCB is used to select promising parent nodes for generation. The selected parents generate child methods using LLM agents, are then evaluated through simulator-based execution, and updated through backpropagation along the selected paths. Panel (b) shows a selected-node generation example where AutoHPFL selects K-means-based piecewise ridge regression (KM-PR) as the parent method, generates PLS-assisted K-means piecewise ridge regression (PLS-KM-PR), and achieves lower PF and V_m MRE in simulator-based evaluation.

Comment 2.3: The formulation of the space mapping ψ is not sufficiently clear. It should be explicitly stated how this mapping is technically implemented to convert code and text into a unified vector space.

***Response to Comment 2.3**:* We thank the reviewer for pointing out that the technical implementation of ψ was unclear. We revised Section III-B to define $\mathcal{M}_i$ as a method-information set and to explain how ψ maps selected method information into a unified vector space. In the general formulation, $\mathcal{M}_i$ can include executable code, textual description, feature-construction schema, parent-child modification records, or other available attributes. The mapping ψ first serializes selected fields of $\mathcal{M}_i$ into a code-text representation and then embeds this representation into a unified vector space using a code-text embedding model `text-embedding-v4` provided by DashScope. The novelty score is computed by the L2-normalized cosine distance between the candidate vector and the method database.

Section III-B. Tree-Level Search with Novelty-aware MCTS

For each candidate method i, we define a method representation $\mathbf{z}_i = \psi(\mathcal{M}_i)$, where $\mathcal{M}_i$ denotes a method-information set, such as executable code, textual description, feature-construction schema, parent-child modification records, or other available attributes. The mapping $\psi(\cdot)$ first serializes the selected fields of $\mathcal{M}_i$ into a code-text representation and then embeds it into a unified vector space using a code-text embedding model. The novelty score is then defined as:

$$s_{novelty} = \min_{j \in \mathcal{B}} \left(1 - \frac{\mathbf{z}_i \cdot \mathbf{z}_j}{\|\mathbf{z}_i\| \|\mathbf{z}_j\|} \right) \tag{7}$$

where $\mathcal{B}$ denotes the method database used by AutoHPFL, which contains more than 50 implemented HPFL/DPFL variants, feature templates, and code interfaces for RAG and novelty evaluation. The seven baselines in Table I are representative methods selected for compact numerical comparison.

We also clarified the concrete implementation used in the experiments.

Section IV-A. Experiment Setup

... $\mathcal{M}_i$ in Section III is set to executable MATLAB code. The serialized code text is embedded using the `text-embedding-v4` model provided by DashScope, and subsequently L2-normalized to facilitate novelty evaluation based on cosine distance.

Comment 2.4: In Fig. 2, the labeling on the horizontal axis is inconsistent . The first bar is labeled as AutoDPFL , whereas the proposed framework is called AutoHPFL throughout the paper. Please correct this typo.

***Response to Comment 2.4**:* We thank the reviewer for identifying this typo. The label "AutoDPFL" in the original Fig. 2 was incorrect and has been corrected to "AutoHPFL." We also checked the manuscript for naming consistency. The revised Fig. 2 is presented:

Section IV-C. Ablation Study

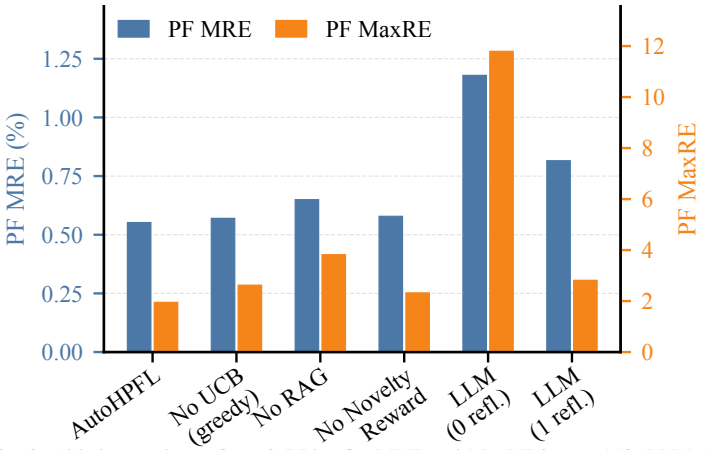

Fig. 2. Ablation study on Qwen3.5-Plus for MRE and MaxRE in case118. LLM (0 refl.) denotes a single foundation model, and LLM (1 refl.) denotes a single foundation model with one reflection.

Response to Reviewer 3

Summary:

This article presents AutoHPFL, an autonomous algorithm discovery framework that employs LLM agents for Physics-data-hybrid-driven power flow linearization (HPFL) methods. Instead of limiting itself to manually crafted feature mappings, AutoHPFL autonomously searches the design space of linearization techniques. Here, agents iteratively refine candidates by focusing on the actual environment execution feedback. Experiments on IEEE 118-bus and 33-bus systems report 44.4% error reduction over baselines, thereby demonstrating substantial improvement over existing baselines.

Pros:

(1) Innovative integration of LLM agents with MCTS. The integrated framework of upper confidence bound selection, RAG-based knowledge retrieval, and tree-based reflection is both an interesting and non-trivial contribution.

(2) The reward formulation is interesting. It penalizes convergence to a narrow family of similar solutions, thereby enabling an efficient exploration and exploitation.

(3) The performance metrics presented table I is promising. For all three benchmarks, the proposed method outperforms the baselines by significant margins.

***Response to the General Comment**:* We thank the reviewer for recognizing the innovation of and the promising performance of AutoHPFL. As for the concerns, we agree that the original manuscript should provide more technical intuition for the reported performance and should more explicitly discuss the reliability concern caused by RAG/LLM hallucination. We revised the manuscript accordingly. Specifically, please refer to:

- **Response to Comment 3.1** for the added technical discussion explaining why HPFL and AutoHPFL contribute to the huge performance boost in Table I, as well as the revised numerical presentation with three independent runs and their arithmetic mean.
- **Response to Comment 3.2** for the added discussion on hallucination risk, simulator-in-the-loop validation, reflection-based regeneration, and future reliability improvements.

Comment 3.1: It has a lack of theoretical intuition. A small paragraph on why HPFL works will be beneficial to the practitioners. Specifically, the huge performance boost in Table I should be supported with technical reasoning and insights.

***Response to Comment 3.1**:* We thank the reviewer for this important suggestion. We agree that the original manuscript mainly reported numerical improvements, but did not sufficiently explain why HPFL and AutoHPFL can achieve such improvements. To address this comment, we revised the manuscript along two complementary directions. On the mechanism side, we added a technical discussion in Section IV-B to explain the performance gains from three aspects: physics-informed feature construction, automated algorithm design, and closed-loop trial-and-refinement. On the evidence side, we revised the numerical presentation by reporting three independent AutoHPFL runs and their arithmetic mean, so that the reported improvement is not tied to a single stochastic LLM-agent trajectory. Specific explanations are provided below.

First, HPFL is more effective than directly fitting raw injection samples because physics-informed features introduce power-system structural information into the linear model. Raw active/reactive injection samples alone do not explicitly encode network topology, electrical coupling, or local operating-region characteristics. As a result, a purely data-driven linear model must infer these relationships mainly from samples. By contrast, HPFL feature construction provides the regression model with more informative inputs, reducing the burden of rediscovering network-induced dependencies from data alone. This explains why physics-data-hybrid feature mappings can improve linearization accuracy in principle.

Second, AutoHPFL further improves HPFL by automating the design of such feature mappings and related algorithmic choices. The improvement is not simply due to a single hyperparameter tuning step. Instead, AutoHPFL converts the reasoning and code-generation capability of LLM agents into an executable trial-and-refinement process. The agents can propose structured modifications to feature construction, partitioning rules, and regression structures, while RAG supplies domain papers, implemented methods, and code templates as external context. This makes the generated candidates more grounded than one-shot LLM outputs based only on parametric knowledge.

Third, AutoHPFL provides an environment in which these candidate methods can be repeatedly tried, executed, evaluated, and corrected. The search tree records previous attempts and their performance, novelty-aware MCTS allocates the search budget to promising yet underexplored branches, and simulator-based evaluation ensures that candidates are selected according to empirical errors rather than textual plausibility. When a candidate fails or performs worse than its parent, tree-informed reflection uses execution logs and cross-branch references to regenerate a revised method. Therefore, AutoHPFL is better understood as a closed-loop algorithm-design framework, rather than a direct LLM-generation baseline. This mechanism explains why the performance gains in Table II are systematic rather than accidental.

The corresponding technical discussion added in Section IV-B is shown below.

Section IV-B Result Analysis:

...

The gain of HPFL mainly comes from physics-informed features, which encode network coupling relationships and help a linear model capture dependencies beyond raw injection samples, reducing the need for the regression model to rediscover such coupling relationships from samples alone. AutoHPFL further improves this process by converting the reasoning and code-generation capability of LLM agents into an executable trial-and-refinement procedure. With RAG-provided domain context and a tree-structured search space, agents can iteratively generate, evaluate, and revise candidate methods. Simulator-based execution ensures candidates are evaluated by empirical performance rather than textual plausibility, and tree-informed reflection turns failed or degraded attempts into correction signals. This closed-loop design also helps mitigate the impact of inaccurate or irrelevant LLM/RAG outputs, since generated candidates cannot be retained without executable validation. Thus, AutoHPFL goes beyond one-shot LLM generation and forms a closed-loop algorithm-design framework, explaining the consistent improvements in Table I.

In addition to the above mechanism-level explanation, we revised the presentation of numerical results to make this conclusion more robust. Instead of reporting only a single AutoHPFL run, Table I now reports three independent runs and their arithmetic mean for each test system. The revised results show that AutoHPFL consistently outperforms

the best non-AutoHPFL baseline across all three systems and both output types. In particular, AutoHPFL reduces PF MRE by up to 35.9% on case300 and V_m MRE by up to 14.7% on case33bw. Moreover, all independent AutoHPFL runs remain below the corresponding best baseline, indicating that the improvement is not due to a single favorable LLM-agent trajectory.

Section IV-B Result Analysis:

Table I reports PF and V_m MRE over three independent AutoHPFL runs, with the Mean row denoting their arithmetic average. Compared with the best non-AutoHPFL baseline in each column, AutoHPFL consistently achieves lower mean errors on all three systems and for both output types. In particular, AutoHPFL reduces PF MRE by up to 35.9% on case300 and V_m MRE by up to 14.7% on case33bw. Moreover, all independent AutoHPFL runs outperform the corresponding best baseline, indicating that the gain is not due to a single favorable LLM-agent trajectory.

TABLE I. PF and voltage-magnitude linearization results (MRE, %).

(a) Branch Active-Power Flow (PF MRE)			
Method	**case118**	**case300**	**case33bw**
LS_LIFX	1.012	1.165	0.757
RR_KPC	0.819†	0.932	0.625
LS_COD	1.397	1.169	0.559†
RR_WEI	2.583	4.464	0.561
PLS_NIP	1.397	1.317	0.559†
PLS_CLS	0.822	0.842†	0.754
DLPF_C	1.466	1.418	0.566
AutoHPFL (3 runs)	0.573 / 0.530 / 0.558	0.587 / 0.528 / 0.502	0.463 / 0.468 / 0.468
AutoHPFL Mean	**0.554**	**0.539**	**0.466**
(b) Voltage Magnitude (V_m MRE)			
Method	**case118**	**case300**	**case33bw**
LS_LIFX	0.793	0.570†	0.770
RR_KPC	0.724	0.876	0.595
LS_COD	0.551†	0.571	0.524†
RR_WEI	2.153	12.067	0.569
PLS_NIP	0.551†	0.660	0.524†
PLS_CLS	0.718	0.798	0.709
DLPF_C	0.612	0.715	0.565
AutoHPFL (3 runs)	0.499 / 0.463 / 0.472	0.562 / 0.510 / 0.451	0.446 / 0.447 / 0.449
AutoHPFL Mean	**0.478**	**0.507**	**0.447**

† denotes the best non-AutoHPFL baseline in each column. AutoHPFL (3 runs) lists three independent runs, and Mean is their arithmetic average. Lower is better.

Comment 3.2: LLM-produced domain knowledge using RAG raises a reliability concern. Like how to maintain good performance in the presence of hallucination error. A few lines on this limitation should be provided.

***Response to Comment 3.2**:* We thank the reviewer for pointing out the reliability concern caused by RAG/LLM hallucination. We agree that retrieved domain knowledge, LLM-generated suggestions, or generated code may be inaccurate, irrelevant, or non-executable. If such outputs were directly adopted as valid knowledge or accepted algorithms, they could indeed affect the reliability of AutoHPFL. To address this concern, we revised the manuscript to clarify that AutoHPFL does not rely on RAG/LLM textual outputs alone. Instead, it uses RAG only as contextual

support and relies on executable simulation, objective error metrics, and tree-informed reflection to filter or correct candidate methods. The specific revisions are as follows.

First, in AutoHPFL, RAG-retrieved content is not treated as verified knowledge or as a final algorithmic conclusion. It is only used as contextual support for LLM-agent candidate generation. Whether a generated candidate method is accepted depends entirely on whether it can pass the standardized code interface, execute in the real computational environment, and achieve competitive objective error metrics. In other words, AutoHPFL does not rely on the textual judgment of LLM/RAG to decide whether a method is good. Instead, it uses simulator-in-the-loop validation to screen candidate algorithms. The corresponding revision in Section III-C is shown below.

Section III-C. Node-Level Generation with Tree-informed Reflection

At the node level, parent nodes selected by the tree-level search are processed in parallel, with each selected node assigned to a dedicated LLM agent to generate an executable child candidate under a standardized interface. The generation is conditioned on the parent method, RAG-retrieved context, elite-node references, and historical search feedback. The RAG module provides external context from domain papers, implemented baseline methods, and code templates, and is used only as generation support rather than verified knowledge. Thus, the agent performs structured refinement of the selected parent method instead of free-form method generation.

Second, we clarified how AutoHPFL handles cases where the retrieved RAG content is inaccurate or mismatched with the current search context. In practice, the agent is not required to copy the retrieved content into the candidate algorithm. It first uses the retrieved information together with the selected parent method, standardized interface, historical feedback, and its own reasoning capability. When the retrieved content is not compatible with the current parent node or the intended modification direction, the agent can fall back to the parent-node structure and implementation template to generate a new child candidate. This mechanism cannot completely eliminate hallucination, but it reduces the risk that irrelevant or incorrect RAG content is directly injected into the candidate method. In all cases, the generated candidate must still pass the same executability check and simulator-based evaluation before it can be retained.

Third, hallucination risk in AutoHPFL is not limited to RAG retrieval. It may also arise from LLM-generated code, invalid interface calls, irrelevant feature construction, or algorithmic modifications that look plausible in text but fail in execution. We therefore added a clearer explanation of the multi-layer safeguards used in AutoHPFL. The RAG database is curated from HPFL/DPFL-related papers, implemented methods, code templates, and feature-construction examples. The generated code must run under a standardized interface, and candidates that fail to execute or violate interface requirements are rejected. If a candidate fails during execution or performs worse than its parent, tree-informed reflection is triggered, and the agent regenerates a revised child candidate using error logs, performance feedback, and elite-node references. In addition, reward computation and backpropagation are based on actual simulation errors and novelty scores, rather than subjective LLM evaluation. The simulator-based validation and reflection mechanism in the revised paper are as follows.

Section III-C. Node-Level Generation with Tree-informed Reflection

...

The generated child code is then executed by a simulator wrapper in a real computational environment. The

wrapper completes necessary execution settings, runs the candidate script, extracts performance metrics from the output, and returns structured error messages when execution fails or times out. During this process, the agent assists with interface checking, log parsing, and result collection, but the final score is computed only from simulator outputs. This simulator-in-the-loop validation prevents LLM-generated candidates from being accepted based on textual plausibility alone.

If execution fails or the child degrades relative to its parent, tree-informed reflection is triggered. The agent diagnoses the failure using execution logs and parent-method structure, and retrieves cross-branch references from low-error and high-novelty elite pools maintained by the search tree. These elite nodes provide examples of effective and diverse designs from other branches. Based on this feedback, the agent regenerates a revised candidate, turning reflection into simulator-grounded error correction and cross-branch knowledge transfer.

Finally, we explicitly acknowledge hallucination as a remaining limitation in the revised Conclusion. Although AutoHPFL mitigates hallucination risk through executable simulator-based validation and reflection-based regeneration, LLM/RAG hallucination may still produce invalid or irrelevant candidates and cannot be fully eliminated by the current framework. Future work will further strengthen reliability through retrieval-source verification, candidate provenance tracking, and formal code validation. The corresponding limitation statement in the revised Conclusion is shown below.

Conclusion:

...A remaining limitation is that LLM/RAG hallucination may still produce invalid or irrelevant candidates. Future work will focus on hallucination control via retrieval-source verification, candidate provenance tracking, and will further explore applying AutoHPFL to other power system tasks like downstream OPF deployment.